TELEPHONE EXCHANGE MANAGEMENT SYSTEM

Complete System Documentation, Audit Prompt & Enhancement Plan

Built with PHP & MySQL | Three-Role Architecture

Prepared for: COCOBOD Presentation

TABLE OF CONTENTS

TABLE OF CONTENTS 2
SECTION 1 — SYSTEM OVERVIEW 4
1.1 What Is the Telephone Exchange Management System? 4
1.2 Core Purpose & Business Value 4
1.3 Three-Role System Architecture 4
SECTION 2 — ROLE DASHBOARDS & FUNCTIONALITIES 5
2.1 Administrator Dashboard 5
2.1.1 Current Administrator Capabilities 5
2.1.2 Administrator Key Metrics (Dashboard KPIs) 5
2.2 Receptionist Dashboard 5
2.2.1 Current Receptionist Capabilities 5
2.2.2 Receptionist Workflow 6
2.3 Lab Technician / Maintenance Dashboard 6
2.3.1 Current Technician Capabilities 6
2.3.2 Technician Workflow 6
SECTION 3 — HOW THE SYSTEM COMMUNICATES 7
3.1 Data Flow Between Roles 7
3.2 Authentication & Session Management 7
3.3 Database Structure (Core Tables) 7
SECTION 4 — AI SYSTEM AUDIT PROMPT 8
4.1 Phase 1 — Comprehensive System Audit Prompt 8
4.2 Phase 2 — Enhancement Implementation Prompt 8
SECTION 5 — COMPREHENSIVE ENHANCEMENT PLAN 10
5.1 Security Enhancements 10
5.2 UI/UX Enhancement Plan 10
5.2.1 Design System 10
5.2.2 Dashboard Layout Enhancements 10
5.3 Administrator Dashboard Enhancements 11
5.4 Receptionist Dashboard Enhancements 11
5.5 Technician Dashboard Enhancements 11
SECTION 6 — IMPLEMENTATION ROADMAP 13
SECTION 7 — PRESENTATION GUIDE FOR COCOBODA 14
7.1 Recommended PowerPoint Structure 14
7.2 Key Talking Points 14
Opening Statement 14
On Security 14

On Scalability 14
On Business Value 15
SECTION 8 — TECHNICAL GLOSSARY 16

SECTION 1 — SYSTEM OVERVIEW

1.1 What Is the Telephone Exchange Management System?

The Telephone Exchange Management System (TEMS) is a web-based platform built using PHP and MySQL, designed to manage, track, and maintain telephone exchange infrastructure for an organisation. The system provides a structured digital environment for logging faults, managing maintenance workflows, coordinating receptionist activities, and giving administrators full visibility and control over all operations.

The system serves as the central hub for telephone infrastructure management — replacing manual, paper-based processes with a streamlined digital platform that supports real-time fault tracking, job card management, role-based access control, and comprehensive reporting.

System Stack

Frontend: HTML, CSS, JavaScript (PHP-rendered) Backend: PHP (server-side logic) Database: MySQL (relational data storage) Architecture: Multi-role web application with session-based authentication

1.2 Core Purpose & Business Value

- Centralise all telephone fault and maintenance records in one system
- Enable role-based access so each user type sees and does only what is relevant to them
- Create a transparent, auditable trail of all faults, repairs, and resolutions
- Reduce response time for fault resolution through organised job assignments
- Give management real-time visibility into the status of all telephone infrastructure
- Provide a production-ready platform suitable for enterprise deployment

1.3 Three-Role System Architecture

The system is built around three distinct user roles, each with a dedicated dashboard, specific permissions, and a defined set of responsibilities:

Role	Primary Responsibility	Access Level
Administrator	Full system control — users, reports, configuration, oversight	Unrestricted — all modules
Receptionist	Front-desk operations — fault logging, customer intake, call tracking	Operational — create & view faults
Lab Technician / Maintenance	Technical work — fault diagnosis, job card management, repair records	Technical — update & resolve faults

SECTION 2 — ROLE DASHBOARDS & FUNCTIONALITIES

2.1 Administrator Dashboard

Role Summary

The Administrator has unrestricted access to all modules. They manage users, configure the system, view all reports, oversee all fault and maintenance records, and have final authority on all system actions.

2.1.1 Current Administrator Capabilities

- User Management — Create, edit, activate, deactivate all user accounts (Receptionist & Technician)
- Role Assignment — Assign and change user roles across the system
- Dashboard Overview — View real-time summary of all faults: open, in-progress, resolved
- Fault Records — View all fault logs submitted by receptionists across all departments
- Job Card Management — View all job cards created by technicians, track completion rates
- Reports & Analytics — Generate reports by date range, fault type, technician, status
- System Configuration — Manage departments, fault categories, priority levels
- Audit Logs — View system activity logs for all user actions
- Notifications — Receive alerts for overdue faults and unresolved issues

2.1.2 Administrator Key Metrics (Dashboard KPIs)

- Total Faults Logged (current period)
- Faults Resolved vs Pending vs In-Progress
- Average Resolution Time by technician
- Most frequent fault categories
- Department with highest fault volume

2.2 Receptionist Dashboard

Role Summary

The Receptionist is the first point of contact for all telephone-related issues. They log incoming fault reports, manage customer interactions, track call records, and coordinate with technicians for fault resolution.

2.2.1 Current Receptionist Capabilities

- Fault Logging — Create new fault reports with details: caller name, department, extension number, fault description, priority level, date and time
- Fault Tracking — View all faults they have logged and monitor their current status
- Call Log Management — Record incoming and outgoing call activities
- Status Updates — Receive updates when a technician changes the status of a logged fault
- Customer Communication — Record customer contact details and correspondence
- Search & Filter — Search fault records by date, status, department, or extension number

- Print/Export — Generate printable fault reports for physical records

2.2.2 Receptionist Workflow

1. Customer reports a telephone fault (in person, call, or email)
2. Receptionist opens a new fault log — fills in all required fields
3. System assigns a unique Fault ID and timestamps the record
4. Fault is visible to the assigned technician immediately
5. Receptionist monitors the fault status until resolved
6. On resolution, receptionist closes the loop with the customer

2.3 Lab Technician / Maintenance Dashboard

Role Summary

The Lab Technician (Maintenance Officer) handles all technical fault diagnosis and resolution. They receive fault assignments, create and manage job cards, update repair progress, and mark faults as resolved.

2.3.1 Current Technician Capabilities

- Fault Assignment View — See all faults assigned to them with full details
- Job Card Creation — Create detailed job cards for each assigned fault: diagnosis, materials needed, estimated time, actions taken
- Status Updates — Update fault status: Pending → In Progress → Resolved → Closed
- Technical Notes — Add technical diagnosis notes and repair documentation
- Parts & Materials — Log materials and parts used during repair
- Completion Reports — Submit completion notes when a fault is fully resolved
- Work History — View their complete history of resolved faults and job cards

2.3.2 Technician Workflow

7. Technician logs in and views their assigned faults dashboard
8. Opens a fault and reviews the receptionist's description
9. Creates a job card: diagnosis, action plan, materials required
10. Updates status to 'In Progress' and begins physical repair work
11. Logs all technical actions and parts used during repair
12. On completion: updates status to 'Resolved', adds completion notes
13. Fault record is updated for admin and receptionist visibility

SECTION 3 — HOW THE SYSTEM COMMUNICATES

3.1 Data Flow Between Roles

The three roles interact through a shared MySQL database. No role communicates directly with another — all information exchange happens through the database and the PHP session layer.

Action	Triggered By	Received By	Data Stored
Log new fault	Receptionist	Technician + Admin	fault_id, description, extension, priority, status, timestamp
Create job card	Technician	Admin	job_card_id, fault_id, diagnosis, materials, actions
Update fault status	Technionist	Receptionist + Admin	fault_id, new_status, updated_at
Resolve fault	Technician	Receptionist + Admin	fault_id, resolution_notes, resolved_at
Generate report	Admin	Admin only	Aggregated records from all tables

3.2 Authentication & Session Management

- Users authenticate via username/password on the login page
- PHP sessions store the logged-in user's ID and role
- Each dashboard page checks the session role before rendering
- Unauthorised access attempts are redirected to the login page
- Session timeout logs users out after a defined period of inactivity

3.3 Database Structure (Core Tables)

The MySQL database underpins all system operations. The core tables are:

Table Name	Purpose & Key Fields
users	Stores all user accounts. Fields: user_id, name, email, password_hash, role, status, created_at
faults	All fault reports. Fields: fault_id, reported_by, department, extension, description, priority, status, assigned_to, created_at, updated_at
job_cards	Technician job records. Fields: job_id, fault_id, technician_id, diagnosis, actions_taken, materials_used, status, created_at, completed_at
call_logs	Receptionist call records. Fields: log_id, caller_name, department, extension, call_type, notes, logged_by, timestamp
notifications	System alerts. Fields: notif_id, user_id, message, type, read_status, created_at
audit_logs	All user actions. Fields: log_id, user_id, action, table_affected, record_id, timestamp

SECTION 4 — AI SYSTEM AUDIT PROMPT

Copy and paste the prompt below into your AI model (Integravity or similar). Run this BEFORE any enhancements. Do not make any changes until the audit report is returned.

INSTRUCTIONS

Paste Phase 1 into the AI first. Wait for the full audit report. Then paste Phase 2. Test before Phase 3. Always one phase per conversation session.

4.1 Phase 1 — Comprehensive System Audit Prompt

TELEPHONE EXCHANGE MANAGEMENT SYSTEM — FULL CODEBASE AUDIT

You are performing a comprehensive audit of the Telephone Exchange Management System (TEMS) — a PHP and MySQL web application with three user roles: Administrator, Receptionist, and Lab Technician / Maintenance Officer.

READ THE ENTIRE CODEBASE FIRST. Do not make any changes. Audit and report only.

AUDIT TASKS:

- Map the complete file and folder structure — every PHP file, CSS file, JS file, and include/config file
- Document every database table: name, all fields, data types, foreign keys, indexes
- Map all three dashboards: every tab, panel, button, form, and table in Administrator, Receptionist, and Technician views
- Trace all authentication logic: login, session management, role-based redirects, logout
- Identify every form and its submit handler: what data is collected, validated, and stored
- Map all database queries: for each query, what data is read/written and which role triggers it
- Identify all existing security implementations: input sanitisation, prepared statements, CSRF protection, password hashing
- Identify all existing UI components: tables, modals, badges, alerts, cards, navigation
- Find all known bugs, broken features, incomplete functions, or TODO comments
- Assess current UI/UX quality: responsiveness, theme consistency, accessibility

OUTPUT FORMAT:

- Section A — File Structure Map (every file and its purpose)
- Section B — Database Schema (all tables with all fields)
- Section C — Dashboard Feature Inventory (by role, every feature)
- Section D — Authentication & Security Assessment
- Section E — Known Issues & Incomplete Features
- Section F — UI/UX Quality Assessment
- Section G — Enhancement Readiness Score (what needs work before enhancements)

4.2 Phase 2 — Enhancement Implementation Prompt

TEMS — ENHANCEMENT IMPLEMENTATION (run after audit)

Based on the audit report above, implement all enhancements in the strict order below. Do NOT break existing functionality. Show BEFORE/AFTER for every file changed.

ENHANCEMENT ORDER:

14. Security hardening first — prepared statements, input validation, CSRF tokens, bcrypt passwords
15. Database enhancements — add missing indexes, foreign keys, audit_logs table if missing
16. UI/UX overhaul — responsive layout, consistent theme, dark/light toggle, modern cards and tables
17. Administrator dashboard — advanced analytics charts, KPI cards, exportable reports, user management improvements
18. Receptionist dashboard — improved fault logging form, real-time status badges, call log enhancements, search and filter
19. Technician dashboard — enhanced job card form, work history timeline, materials tracking, status update workflow
20. Notification system — in-app notification bell for all roles, real-time status change alerts
21. Reporting module — PDF and Excel export, date-range filters, department and technician performance reports
22. Audit trail — comprehensive logging of all user actions with timestamps and IP addresses
23. Final QA — test all three roles end-to-end, verify no existing functionality is broken

SECTION 5 — COMPREHENSIVE ENHANCEMENT PLAN

5.1 Security Enhancements

Priority: CRITICAL
Security must be implemented before any feature enhancements. A production-ready system must be secure by default.

Enhancement	Implementation Detail	Priority
Prepared Statements	Replace all raw MySQL queries with PDO prepared statements to prevent SQL injection	Critical
Password Hashing	Use PHP password_hash() with bcrypt for all password storage and verification	Critical
CSRF Protection	Add CSRF token to all forms — generate token in session, verify on submit	Critical
Input Sanitisation	Sanitise all user inputs server-side using htmlspecialchars() and filter_var()	Critical
Session Security	Regenerate session ID on login, implement session timeout, add IP binding	High
Role Enforcement	Server-side role check on every protected page — no client-side-only guards	Critical
HTTPS Enforcement	Force HTTPS via .htaccess redirect, add HSTS header	High
Brute Force Protection	Implement login attempt limiting — lock account after 5 failed attempts	High

5.2 UI/UX Enhancement Plan

5.2.1 Design System

- Implement a consistent colour palette and typography system across all three dashboards
- Standardise component styles: buttons, forms, tables, cards, badges, alerts, modals
- Add light mode / dark mode toggle with localStorage persistence
- Ensure full responsiveness for desktop, tablet, and mobile screens
- Implement a fixed left sidebar navigation with role-specific menu items and active state indicators
- Add animated page transitions and loading states for all data fetches

5.2.2 Dashboard Layout Enhancements

- Top header bar: system logo, logged-in user name and role, notification bell, profile dropdown
- KPI summary cards at the top of each dashboard with relevant metrics and trend indicators
- Advanced data tables: sortable columns, pagination, column filters, bulk actions, CSV/PDF export
- Status badges with colour coding: Pending (yellow), In Progress (blue), Resolved (green), Closed (grey)

- Interactive charts and graphs in the admin dashboard (bar chart for fault volume, pie for categories, line for trends)
- Timeline view for fault history: visual step-by-step progress from logged to resolved
- Modal dialogs for all create/edit forms — no separate page navigations for small actions

5.3 Administrator Dashboard Enhancements

Feature	Enhancement Detail
Analytics Dashboard	Interactive charts: fault volume by month, resolution rates, technician performance, department breakdown. Use Chart.js
User Management	Full CRUD table with search, filter by role/status, bulk activate/deactivate, activity log per user
Advanced Reports	Date range picker, filter by technician/department/status, export to PDF and Excel
System Settings	Manage fault categories, departments, priority levels, SLA targets per category
SLA Monitoring	Flag faults that have exceeded their SLA (Service Level Agreement) resolution time
Audit Trail	Full log of all user actions with timestamp, IP address, and affected record
Announcement Board	Admin can post announcements visible to all users on their dashboards

5.4 Receptionist Dashboard Enhancements

Feature	Enhancement Detail
Fault Logging Form	Enhanced form with auto-suggest for department names, extension number validation, priority urgency indicator, file attachment for evidence photos
Live Status Tracking	Colour-coded status badges on fault list, real-time status updates without page refresh (AJAX polling)
Call Log Enhancements	Call duration field, call outcome dropdown (resolved/escalated/callback), quick search by caller name or extension
Customer Lookup	Quick search to find a customer's fault history by name or extension number
Notification Centre	In-app notifications when a technician updates a fault status the receptionist logged
Daily Summary Widget	Today's faults logged, pending faults, resolved today — at a glance at the top of the dashboard

5.5 Technician Dashboard Enhancements

Feature	Enhancement Detail
Job Card Form	Structured form: fault diagnosis section, step-by-step actions taken, materials/parts used with quantity, estimated vs actual repair time

Feature	Enhancement Detail
Work Queue	Priority-sorted list of assigned faults with urgency badges, overdue highlights, and quick status update buttons
Fault Timeline	Visual progress timeline for each fault: Logged → Assigned → In Progress → Resolved → Closed
Materials Log	Track parts and materials used per job — supports inventory awareness and cost reporting
Performance Stats	Personal stats: faults resolved this week/month, average resolution time, completion rate
Completion Report	Structured completion form: root cause, fix applied, preventive recommendation, follow-up required (yes/no)

SECTION 6 — IMPLEMENTATION ROADMAP

The enhancement plan is structured into four implementation phases. Each phase builds on the previous one without breaking existing functionality.

Phase	Name	What Gets Built	Est. Effort
Phase 1	Security Foundation	Prepared statements, bcrypt passwords, CSRF tokens, session hardening, role enforcement	1-2 days
Phase 2	UI/UX Overhaul	Design system, responsive layout, sidebar nav, KPI cards, status badges, dark mode toggle	2-3 days
Phase 3	Feature Enhancement	All dashboard enhancements per role, advanced tables, job card improvements, notification system	3-4 days
Phase 4	Reporting & Analytics	Admin analytics charts, PDF/Excel export, SLA monitoring, audit trail, announcement board	2-3 days

Important Rule

Always audit first. Never skip Phase 1 (Security). Test every phase before moving to the next. Never paste multiple phases into the AI at once.

SECTION 7 — PRESENTATION GUIDE FOR COCOBODA

7.1 Recommended PowerPoint Structure

Slide	Title	Content Summary
1	Title Slide	System name, your name, date, tagline: 'A production-ready telephone infrastructure management platform'
2	The Problem	What problems does the system solve? Manual fault tracking, no visibility, slow resolution, no accountability
3	The Solution	TEMS overview — what it is, what it does, who uses it, what technology it's built on
4	System Architecture	Three-role diagram: Admin / Receptionist / Technician — their responsibilities and how they connect
5	Administrator Dashboard	Screenshot + 4-5 bullet points of key features and KPIs
6	Receptionist Dashboard	Screenshot + fault logging workflow, call log management
7	Technician Dashboard	Screenshot + job card workflow, resolution timeline
8	Fault Lifecycle	Visual: from customer report → logged → assigned → in progress → resolved → closed
9	Security Features	List key security implementations — shows production-readiness
10	Reports & Analytics	Show admin charts, export capability, SLA monitoring
11	Enhancement Roadmap	4-phase plan — what has been built and what can be added
12	Why TEMS for Cocoboda	Business value: efficiency, accountability, real-time visibility, scalability
13	Live Demo / Q&A	Transition to live system demonstration

7.2 Key Talking Points

Opening Statement

"The Telephone Exchange Management System is a fully operational, role-based web application built in PHP and MySQL. It replaces manual fault tracking with a structured digital platform that gives every stakeholder — from the front desk to management — exactly the visibility and tools they need to do their job efficiently."

On Security

"Every aspect of the system is built with production-grade security in mind — prepared statements against SQL injection, bcrypt password hashing, CSRF protection on all forms, and server-side role enforcement on every page."

On Scalability

"The system is built to scale — new user roles, departments, or fault categories can be added through the admin panel without touching code. The database schema supports growth, and the architecture is clean enough to add an API layer or mobile interface in a future phase."

On Business Value

"Before this system, fault tracking was manual — paper forms, phone calls, no visibility. Now, every fault has a digital audit trail from the moment it's reported to the moment it's resolved. Management can see, in real time, what is open, who is working on it, and how long it's taking."

SECTION 8 — TECHNICAL GLOSSARY

Term	Definition
PHP	Server-side scripting language used to handle all backend logic, form processing, and database interactions
MySQL	Relational database management system used to store all system data including users, faults, job cards, and logs
PDO	PHP Data Objects — a secure database access layer that uses prepared statements to prevent SQL injection
CSRF Token	Cross-Site Request Forgery token — a security measure that prevents malicious websites from submitting forms on behalf of a logged-in user
bcrypt	A secure password hashing algorithm used to store passwords safely — original password cannot be recovered from the hash
Session	Server-side storage of the logged-in user's identity and role, used to control access throughout the application
Role-Based Access	A security model where each user's access is determined by their assigned role — Admin, Receptionist, or Technician
Job Card	A structured record created by a technician documenting their diagnosis, actions taken, materials used, and resolution for a specific fault
SLA	Service Level Agreement — a defined maximum time within which a fault of a given priority should be resolved
Audit Trail	A chronological log of all system actions — who did what, when, and to which record
AJAX	Asynchronous JavaScript and XML — used to update parts of the page (e.g. fault status) without a full page reload
KPI	Key Performance Indicator — a measurable value used to evaluate system performance (e.g. average resolution time)

TELEPHONE EXCHANGE MANAGEMENT SYSTEM

Complete System Documentation | Audit Prompt | Enhancement Plan | Presentation Guide

Ready for Cocoboda Presentation